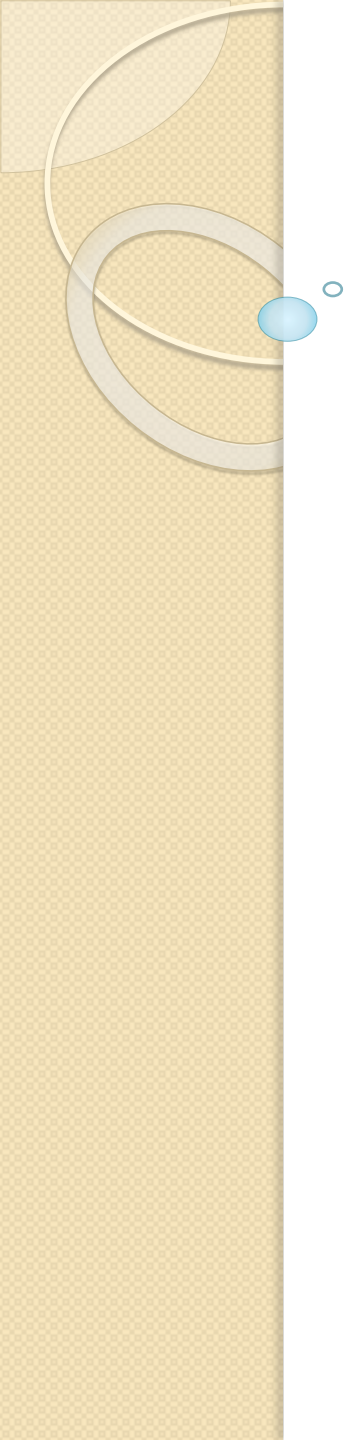




Chapter-4

Logical Database Design

Prepared by: Marta G. (MSc.)

Contents

- Functional dependency
 - Definition & Notation
 - Types
- Normalization
 - Definition & Purpose
 - Normalization types
- Anomalies
 - Types

Functional Dependency

Definition:

- A functional dependency is a **constraint** between **two attributes** or sets of attributes **of entities** in a database.
- Or we can say it is a constraint **that specifies the relationship of one attribute to another attribute in a relation** or a table.
- A functional dependency $\rightarrow$ is **denoted** by $X \rightarrow Y$, between two sets of attributes X and Y that are subsets of relation R .
- It specifies a constraint on the possible tuples that can form a relation state r of R .
- The constraint is that, the **values of Y component of a tuple in r depend on, or are determined by the values of X component.**
 - Alternatively, the **values of X component of a tuple uniquely (or functionally) determine the values of Y component.**

Cont...

Functional dependency Notation/representation:

- **Notation:** $X \hat{=} Y$ ('x' determines 'Y', 'Y' depends on 'X', 'Y' is determined by 'X', the values of 'Y' are determined by or depends on the values of 'X')
- $X \hat{=} Y = X \rightarrow Y$
- The left-hand side attributes determine the values of attributes on the right-hand side.

Cont...

- The essence of functional dependency is that;
- If we say for **the existence of** something, call it **B**, as **primary condition A must exist**, and have a certain value, then we say that $\rightarrow$ "**B is functionally dependent on A**".
- We also often express this idea by saying that:
 - **A determines B**, or
 - **B is a function of A**, or
 - **A functionally governs B**.
- A functional dependency is a **property** of the **semantics/ meaning** of the **attributes**.
- The main **use** of functional dependencies is to **describe further a relation schema R** by **specifying constraints on its attributes** that must hold at all times.

Example I

A	B
01	Melona
02	Alif
03	Haleta
04	Ruth
05	Ribka

A → B

VALUE OF A DETERMINES VALUE OF B

Eg: 03 determines Haleta

- A **determines** B or B is **determined by** A

So that, **A** is called **Determinant** Attribute and **B** is called **Dependent** attribute

Other examples

- **SSN ^a ENAME** → the value of an employee's social security number (SSN) uniquely determines the employee name (ENAME).
- **PNUMBER ^a {PNAME, PLOCATION}** → the value of a project's number (PNUMBER) uniquely determines the project name (PNAME) and location (PLOCATION).
- **{SSN, PNUMBER} ^a HOURS** → a combination of SSN and PNUMBER values uniquely determines the number of hours the employee works on the project per week (HOURS).
- Alternatively, we can say that ENAME is functionally determined by (or functionally dependent on) SSN ...

Roll_No	Stud_Name	Dept_Name	Dept_Building
1	Hana	SE	B1
2	Fuad	CS	B2
3	Yebeltal	SE	B1
4	Biruk	CS	B2
5	Elsa	CE	B3
6	Hermila	Arch	B3

- Let's check the functional dependencies are valid or not?

A) **Similar determinant with similar dependent values**

eg: Dept_name $\rightarrow$ Dept_building (Is this valid??)

SE $\rightarrow$ B1

SE $\rightarrow$ B1

- Here, there is a data redundancy but the data presented in the relation is correct. So there is a **valid functional dependency** b/n dept_name and dept_building
- Hence, if there are **similar determinant** with **similar corresponding dependent values** in the relation, the functional dependency is **valid**.

Roll_No	Stud_Name	Dept_Name	Dept_Building
1	Hana	SE	B1
2	Fuad	CS	B2
3	Yibeltal	SE	B1
4	yibeltal	CS	B2
5	Elsa	CE	B3
6	Hermila	Arch	B3

B) Different determinant with similar dependent values

eg: Dept_name → Dept_building (Is this valid??)

CE → B3

Arch → B3

Or Roll_no → Stud_name (Is this valid??)

3 → yibeltal

4 → yibeltal

- These both are **valid**. Hence, in this case, there is a **valid functional dependency**.
- So we can say that there is dept_name determines dept_building or stud_name is determined by roll_no.

Roll_No	Stud_Name	Dept_Name	Dept_Building
1	Hana	SE	B1
2	Fuad	CS	B2
3	Yibeltal	SE	B1
4	yibeltal	CS	B2
5	Elsa	CE	B3
6	Hermila	Arch	B3

C) Different determinant value with different dependent values

Eg: Roll_No → Dept_name (Is this valid??)

1 → SE

6 → Arch

- Here, we can uniquely identify Dept_name just by using roll number. so that, this case **is valid**
- There is a **valid functional dependency** between such type of attributes

Roll_No	Stud_Name	Dept_Name	Dept_Building
1	Hana	SE	B1
2	Fuad	CS	B2
3	Yibeltal	SE	B1
4	yibeltal	CS	B2
5	Elsa	CE	B3
6	Hermila	Arch	B3

D) Similar determinant value with different dependent values

Eg: Stud_name → Dept_name (Is this valid??)

yibeltal → SE

yibeltal → CS

- Here, we cant uniquely identify Dept_name just by using students name. so that, this case is **invalid**
- There is **an invalid functional dependency** between such type of attributes

Types of Functional Dependency

- Trivial functional dependency
- Non-Trivial functional dependency
- Multi-valued functional dependency
- Transitive functional dependency
- Fully functional dependency

Trivial functional dependency

- Occurs when **dependent attribute** is a **subset** of **determinant attributes**

Stud_Id	Name	Age
001	Liyat	18
002	Yostina	17
003	Herani	19
004	Joye	18
005	Yostina	20

- Eg: {stud_id, Name} $\rightarrow$ Name Is it valid functional dependency?

Yes

- We can get any thing new from this dependency, its useless.

Non-Trivial functional dependency

- If a **dependent attribute** is **not a subset** of **determinant attribute**.

Stud_Id	Name	Age
001	Liyat	18
002	Yostina	17
003	Herani	19
004	Joye	18
005	Yostina	20

- Eg: {stud_id, Name} $\rightarrow$ Age Is it valid functional dependency?

yes

- Here age is not a subset of determinant attributes
- We can find something new by using determinant attributes that we don't already know

Multi-valued functional dependency

- Let say $A \rightarrow \{B, C\}$

This is Multi-valued functional dependency if there is **no functional dependency between dependency B and C** ($B \rightarrow C$ (not valid) and $C \rightarrow B$ (not valid)).

Stud_Id	Name	Age
001	Liyat	18
002	Yostina	17
003	Herani	19
004	Joye	18
005	Yostina	20

Eg: $\text{Stud_id} \rightarrow \{\text{Name}, \text{Age}\}$

Can we determine Age by using name ($\text{Name} \rightarrow \text{Age}$)? Or can we determine name by using age ($\text{Age} \rightarrow \text{Name}$)? **NO**

So, there is Multi-valued functional dependency between these attributes.

Partial Dependency

- If an **attribute** which is **not** a **member** of the **key** is **dependent** on **some part of the key** (if we have **composite/candidate key**) then that attribute is partially functionally dependent on the key.
- **Let** $\{A,B\}$ is the key set (candidate keys) and C is no key attribute. Then if $\{A,B\} \rightarrow C$ and $[B \rightarrow C \text{ or } A \rightarrow C]$ holds
- We say $\rightarrow C$ is **partially functionally dependent on $\{A, B\}$**
- **Note:** Partial dependency **occurs** when a **non-prime attribute** is **functionally dependent** on **part** of a candidate/composite **key**.
- If there is **one candidate key $\{A,B\}$** , all attribute present on those candidate keys are called **prime attributes**. Only A from the set is **not** considered as candidate key but a **part of candidate key**.
- **Part of candidate key** is still **prime attribute**

Cont...

- When part of candidate key can determine non prime attribute, it said to be partial dependency occurs.

Part of candidate key $\rightarrow$ Non prime attribute

- When non prime attribute can only determined by complete candidate key then it said to be full functional dependency exists.

complete candidate key $\rightarrow$ Prime attribute

Example

Lecturer_Id	Course_code	Lecturer_Name	Course_name
3010	SEng2012	Senait	Programming I
3015	SEng4021	Marta	Database
3020	SEng6022	Abey	OOP
3022	SEng5131	Sophia	Requirement Engineering

- **Lecturer_id** is **unique ID** for **lecturer** and **course_code** is unique ID for **course name**.
 - **Candidate keys:** {Lecturer_id , course_code } so these are prime attributes
 - Lecturer_Name and Course_name both are non-prime attributes.
- If {Lecturer_id , course_code } $\rightarrow$ Lecturer_Name and
{Lecturer_id} $\rightarrow$ Lecturer_Name, these indicates **partial dependency**.

Cont...

Steps

1. Identify candidate keys
2. Differentiate prime and non prime attributes
3. Find a non- prime attribute that is determined by a part or proper subset of candidate key

Cont...

Transitive Dependency :

- In mathematics and logic, a transitive relationship is a relationship of the following form:
 - **Example:** If $X > Y$ and $Y > Z$ then $X > Z$.
- “If $A \rightarrow B$ is **valid** functional dependency, and if also $B \rightarrow C$ is **valid** functional dependency, then $A \rightarrow C$ according to the rule of transitive dependency.”
- Generalized way of describing transitive dependency is that:
 - If **A** functionally governs **B**, **AND**
 - If **B** functionally governs **C** **THEN**
 - **A** functionally governs **C**, provided that **neither C nor B determines A**
i.e. $(B \not\rightarrow A \text{ and } C \not\rightarrow A)$
- In the normal notation: $\{(A \rightarrow B) \text{ AND } (B \rightarrow C)\} \Rightarrow A \rightarrow C$ provided that $B \not\rightarrow A$ and $C \not\rightarrow A$

Example

Emp_Id	Emp_Name	Dept_Name	Dept_Building
1010	Temnit	HR	B10
1224	Bethelhem	Finance	B11
1334	Gion	CS	B12

- $\text{Emp_Id} \rightarrow \text{Dept_name}$ $\text{Dept_name} \rightarrow \text{Dept_Building}$
- Both are **valid** functional dependencies
- So that, by transitivity **Emp_Id determines Dept_Building**
 $\text{Emp_Id} \rightarrow \text{Dept_Building}$
- This is **indirect functional dependency** which is called **transitive functional dependency**.

Fully functional dependency

- An attribute is fully functionally dependent on another attribute, **if it is functionally dependent** on a set of **attributes** and **not on any of its proper subset**.
- Let $\{A,B,C\}$ is the key set and D is not key attribute.
- $\{A,B,C\} \rightarrow D$, D is **fully functionally dependent** on the **determinants** if it can't be determined by any of the determinants proper subset (A, B, C, AB, BC, AC). Or if

$A \rightarrow D$ **X**

$AB \rightarrow D$ **X**

$B \rightarrow D$ **X**

$BC \rightarrow D$ **X**

$C \rightarrow D$ **X**

$AC \rightarrow D$ **X**

- Then if $\{A,B,C\} \rightarrow D$ and $[B \rightarrow D, A \rightarrow D, \text{ and } C \rightarrow D]$ doesn't hold
 - (i.e; if A can't determine D and B can't determine D and C can't determine D), then **D is fully functionally dependent on $\{A, B, C\}$**

Fully functional dependency

Stud_Id	Course_code	Marks
01	SEng100	95
01	SEng101	90
02	SEng100	85
02	SEng101	87
03	SEng100	80
03	SEng101	81

{Stud_Id, Course_code} $\rightarrow$ Marks **Valid**

Stud_id $\rightarrow$ Marks **invalid**

Course_code $\rightarrow$ Marks **invalid**

- Hence, mark is fully functionally dependent on both Stud_Id and Course_code since it can be determined by neither stud_id nor course_code

Database Anomalies

- **Database Anomalies:** are the flaws in databases which **occurs because of poor database design and storing everything in a single flat database.**
 - These are problems that are **caused** usually by **redundant data.**
 - **Data Redundancy:** occurs when the **same data is stored in more than one place** in the database.
 - Types of database anomaly
 - Insert anomaly
 - Deletion anomaly
 - Update anomaly

Cont...

- **Insertion Anomaly:** Occurs when the **entire primary key is not known** and the **database cannot insert a new record properly**.
 - This would violate entity integrity.
 - Can be **avoided** by using a **sequence number** for the **primary key**

Example

Emp_id	Emp_name	Age	Dept_No	Dept_name	Dept_Phone	Dept_building
001	Senait	22	3	IS	011433	B1
002	Sophia	23	3	IS	011433	B1
003	Abey	24	4	CS	011566	B2
004	Walelign	25	5	IT	011778	B3
005	Belsty	26	6	SE	011999	B4

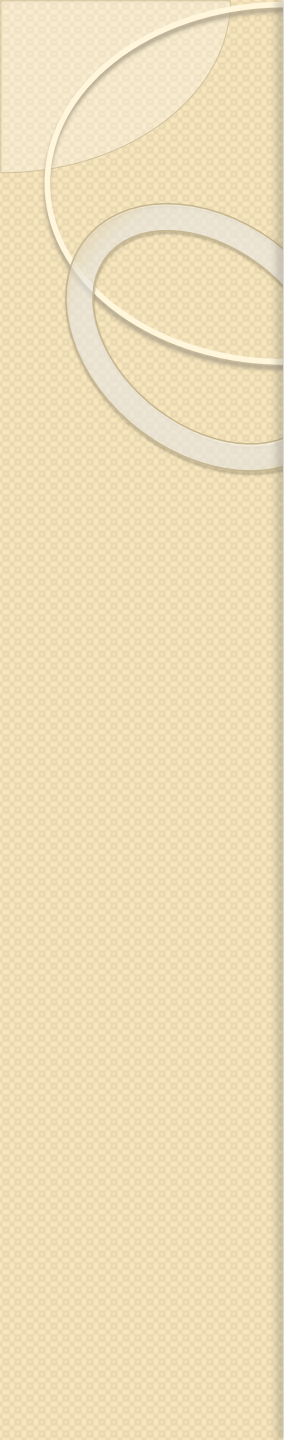
- Employee and department information stored in the same relation called computing
- Let say New department called information science is added to computing college but no employee is hired yet. **Is that possible????**

Cont...

Computing

Emp_id	Emp_name	Age	Dept_No	Dept_name	Dept_Phone	Dept_building
001	Senait	22	3	IS	011433	B1
002	Sophia	23	3	IS	011433	B1
003	Abey	24	4	CS	011566	B2
004	Belsty	25	5	IT	011778	B3
005	Kebede	26	6	SE	011999	B4
????????			7	Information Science	011222	B5

- **NO** because in order to insert the new department detail, at least one employee needs to be recruited since Emp_Id is primary key
- Primary key cant be null, so we cant store department detail till new employee hired
- In such a case **Insertion anomaly occurred**

- 
- **Deletion Anomaly:** happens when a **record is deleted** that **results** in the **loss of other related data**.
 - Can be **avoided by normalizing tables** in a database to minimize their dependency.

Example

Computing

Emp_id	Emp_name	Age	Dept_No	Dept_name	Dept_Phone	Dept_building
001	Senait	22	3	IS	011433	B1
002	Sophia	23	3	IS	011433	B1
003	Abey	24	4	CS	011566	B2
004	Belsty	25	5	IT	011778	B3
005	Kebede	26	6	SE	011999	B4

- Let's say one employee (kebede) from SE dep't decides to left the job, so we need to delete the information from the database or we need to apply delete query like

DELETE FROM computing **WHERE** Emp_Id = 005;

This will **delete the entire row**. Which means we lose the respective department detail/ we **have no information (dept_no, Dept_phone,etc) left about SE department** since he was the only employee in that department

- This case is called **Deletion anomaly**

Cont...

- **Update Anomaly:** Occurs when a change to one attribute value causes the database to either contain inconsistent data or causes multiple records to need changing.
 - May be prevented by making sure tables are in third normal form

Example I

Computing

Emp_id	Emp_name	Age	Dept_No	Dept_name	Dept_Phone	Dept_building
001	Senait	22	3	IS	011433	B1
002	Sophia	23	3	IS	011433	B1
003	Abey	24	4	CS	011566	B2
004	Belsty	25	5	IT	011778	B3
005	Kebede	26	6	SE	011999	B4

- Let say **dept_phone no** is **changed** to 011222 for **IS department**. In this case we need to perform **updatation** for **both** employees(rows) and we can make it **simply** since this table is very small.

What if there is a number of relations?

- Updating on some part and left other create inconsistency
- In such a case there is **updatation anomaly**

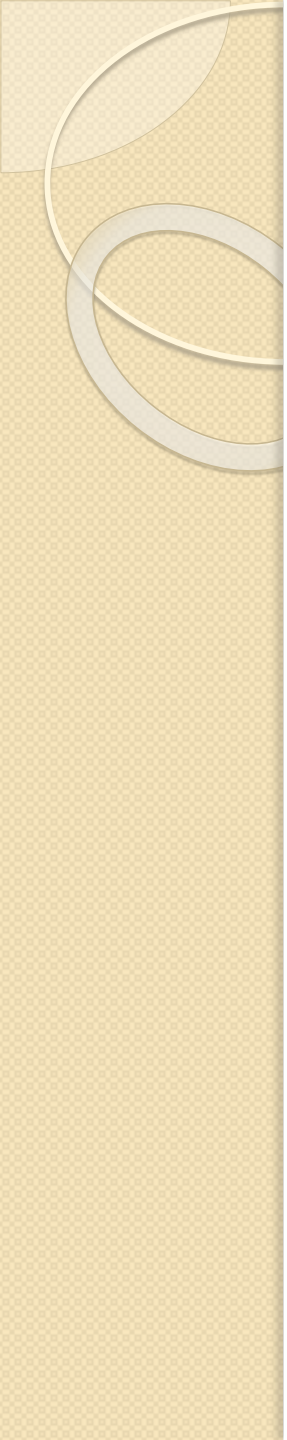
Example 2

Result	Stud_name	Dept_Name	Mark
	Kidist	SE	84.5
	Maki	CS	90
	Kidist	SE	79
	Abis	SE	80

- As you can see, 1st and 3rd rows are similar
- If there is one mark correction for kidist on result relation and you need to update first row, you have to use update query like **update result set mark = 85.5 where stud_name = 'kidist';**
- Then, **both records will be updated**, but updating on 3rd row makes the mark wrong and this affect the database
- In this case, there is **updation anomaly**

Question

- Lets classify computing table into Employee table (Emp_id, Emp_name and Age) and department table (Dept_no, Dept_name, Dept_phone and dept_building)
- There is a r/p b/n the relations and primary key (i.e Dept_id) of one relation will be foreign key of the other
 - I. Does insertion anomaly occurs?
 - II. Does updation anomaly occurs?
 - III. Does deletion anomaly occurs?



**To reduce or remove data redundancy or To
avoid such anomalies, The solution is
Normalization (Reduction or Removal of
duplicate data**

Normalization

- It is a database design technique/concept that **eliminates undesirable characteristics** such as (**Insertion, Update and Deletion anomalies/ problems** which is occurred by **unorganized or redundant data**).
- Normalization is the **transformation of complex user views and data stores to a set of smaller, stable, and easily maintainable data structures**.
- Normalization **divides larger tables into smaller tables and links** them using **relationships**.
- **Purpose:** the purpose of normalization in SQL is to **eliminate or remove redundant (repetitive) data** and **ensure data is stored logically**.

Cont...

- In **relational databases**, normalization is a process that **helps to**
 - Eliminate redundancy,
 - Organize data efficiently,
 - Reduce the potential for anomalies during different database operations, and
 - Improve data consistency.

Normal Form Types



First Normal Form (1NF)

- First normal form (1NF) → is now considered to be part of the **formal definition** of a **relation** in the **basic** or the **flat relational model**.
- A **relation** (table) R is in **1NF** if and only if **all** underlying **domains** of **attributes** contain **only atomic** (simple or indivisible) **values**.
 - **i.e.** The **value of any attribute** in a tuple (row) **must be** a **single value** from the **domain** of their attribute.
- It is defined to **disallow multivalued attributes** (like **Mobile_no, college_degree**), **composite attributes** (like **name, address**), and **their combinations**.

Cont....

- ✓ In other words, 1NF **disallows relations within relations** or **relations as attribute values within tuples**.
- ✓ **Remove** repeating groups (values in a single field).
- ✓ The **primary key with repeating group** attributes are **moved** into a **new table**.
- ✓ **When a relation contains no repeating groups, it is in first normal form(1NF).**

Example I

Is the following relation in 1st normal form? Why?

Id	Name	Phone_no
01	Yabita Getu	091276,098811
02	Haleta Gashaw	096755,076754
03	Temnit Gebru	076543

What about this one?

Id	First_name	Last_name	Phone_no	Alternate_no
01	Yabita	Getu	091276	098811
02	Haleta	Gashaw	096755	076754
03	Temnit	Gebru	076543	Null

- This is **in first normal form** since multivalued and composite attributes are eliminated.
- **If another person has more than 2 phone_no.s** like 3, then phone_no will be classified into 3 attributes

Example 2

- Is this in 1st normal form? If not, make it in 1st normal form

Resident_Id	Address
022	Debre Berhan, 07, 1012
033	Debre zeit, 01, 1505

No

It's **first normal form** is looks like the following

Resident_Id	City	Kebele	House_no
022	Debre berhan	07	1012
033	Debre Zeit	01	1505

Example 3

Relation A

Stu_id	Stud_name	University	Phone_no
111	Abebe	DBU	0945, 0909, 0734
222	Kebede	AAU	0976
333	Almaz	BDU	0734, 0715

- We can change the above relation into the following 1st normal form in the following way

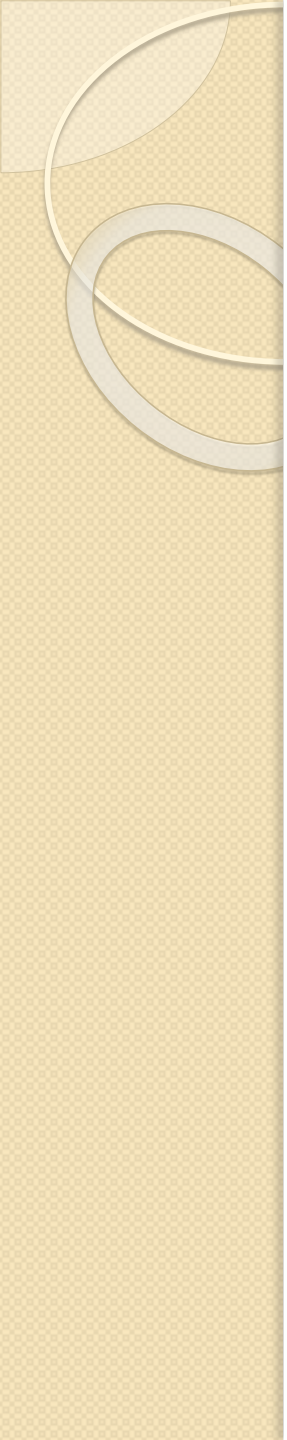
Relation B

Stu_id	Stud_name	University
111	Abebe	DBU
222	Kebede	AAU
333	Almaz	BDU

Relation C

Id	Phone_no	Stud_id
1	0945	111
2	0909	111
3	0734	111
4	0976	222
5	0734	333
6	0715	333

- Where the primary key in relation B is the foreign key for relation C

- 
- **Generally, 1st normal form:**
 - Must not contain composite attribute
 - Must not contain Multi valued attribute

Second Normal Form (2NF)

- Second normal form (2NF) is **based on** the concept of **full functional dependency**.
- For the relation to be in second normal form
 - It should be in **1st normal form** and
 - It should **not** have any **partial dependencies** (all non-key attributes are fully functional dependent on the primary key)
- **Remove** any **partially dependent attributes** and **place them in another relation**.
- A partial dependency is when the attribute(s) is/are dependent on a part of candidate key.
- **Any attributes depends only** on a part of **primary key**

Cont....

Example1: Consider a relation schemas for **Employees and Teams** in a single relation as follows

Emp-Teams(EmpId, Name, BDate, Gender, TeamId, Project, TeamName)

- Then upon decomposition we will have
 - $\text{EmpId} \rightarrow \{\text{Name}, \text{BDate}, \text{Gender}\}$
 - $\text{TeamId} \rightarrow \{\text{Project}, \text{TeamName}\}$
 - Employees(EmpId, Name, BDate, Gender)
 - Teams(TeamId, Project, TeamName)
 - Emp-Teams(EmpId, TeamId)

Example2

Cont...

Stud_id	Stud_name	address

Student

Sub_id	Sub_name

Subject

Score_id	Stud_id	Sub_id	Marks	Teachers
1	002	01		
2	005			
3	002	01		

Example 3

- For example, kaldis coffee has many branch's in Ethiopia. A number of customers can be served/ order cakes, etc./ where ever they want.

Purchase_detail

<u>Customer_Id</u>	Store_Id	Purchase_Location
1	1	Sarbet
1	3	Megenagna
2	1	Sarbet
3	2	Debre_zeit
4	3	Megenagna

- This table has a composite primary key [Customer_ID, Store_ID]. The non-key attribute is [Purchase_Location]. In this case, [Purchase_Location] only depends on [Store_ID], which is only part of the primary key. Therefore, this table does not satisfy second normal form.

Cont...

Purchase

Customer_Id	Store_Id
011	1
011	3
022	1
033	2
044	3

Store

Store_Id	Purchase_location
1	Sarbet
2	Megenagna
3	Debre zeit

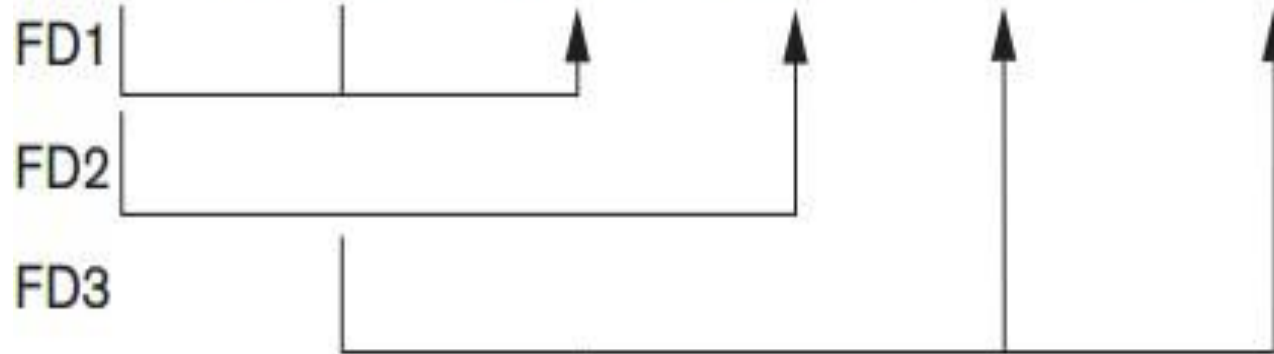
What we have done is to remove the partial functional dependency that we initially had. Now, in the table [TABLE_STORE], the column [Purchase Location] is fully dependent on the primary key of that table, which is [Store ID].

Example2

Cont...

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

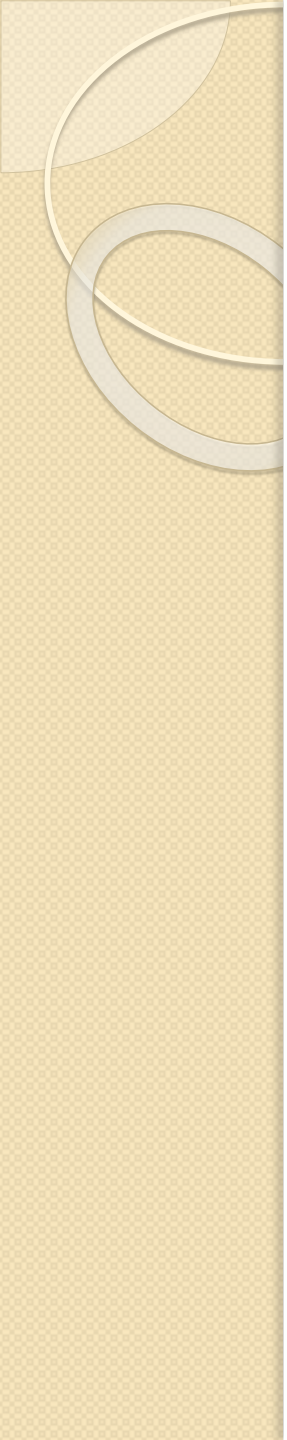
<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------





Third Normal Form (3NF)

- A relation schema R is in 3NF if it satisfies 2NF and **no non-prime attribute of R is transitively dependent on the primary key.**
- A relation schema R is in third normal form (3NF) **if every non-prime attribute of R meets both of the following conditions:**
 - It is **fully functionally dependent** on every key of R.
 - It is **non-transitively dependent** on every key of R.
- **Note:** when we want to design our relation schemas, we have to be sure that they have **neither partial nor transitive dependencies** because these types of dependencies cause the update anomalies.

Cont....

- **Example:** Suppose a company wants to store the complete address of each employee, and the company manager creates a table named employee-details that looks like this:

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

cont...

Analyzing the table

- **Keys:** {{emp-id}, {emp-id, emp-name}, {emp-id, emp-name, emp-zip}}....so on
- **Candidate Keys:** {emp-id, emp-zip}
- **Non-prime attributes:** all attributes **except emp-id and emp_zip** are non-prime as they aren't part of the candidate keys.
- Here, {**emp-state, emp-city & emp-district**} are **dependent** on **emp-zip**, and **emp-zip** is dependent on **emp-id** that makes **non-prime attributes** (emp-state, emp-city & emp-district) **transitively dependent** on **key (emp-id)**.
- This **violates** the rule of 3NF.
- To make this table complies with 3NF **we have to break the table into two tables to remove the transitive dependency.**

Cont...

After normalization

employee table:

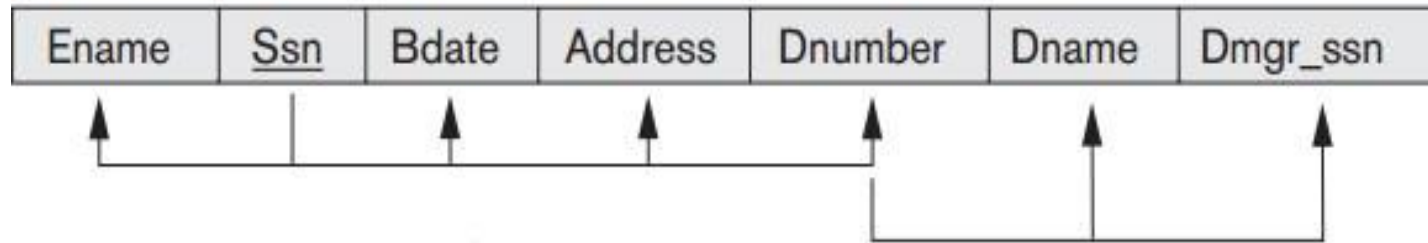
emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

employee_zip table:

emp_zip	emp_state	emp_city	emp_district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

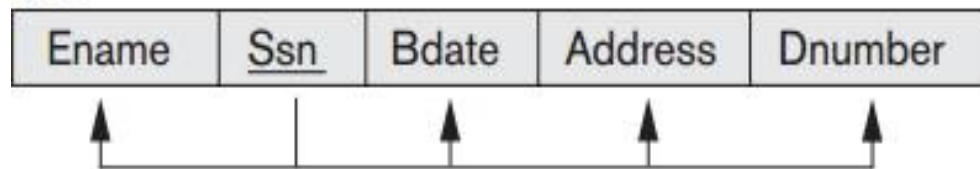
Cont...

EMP_DEPT

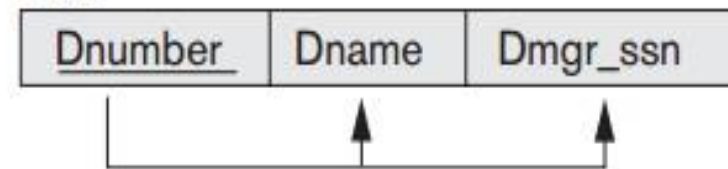


3NF Normalization

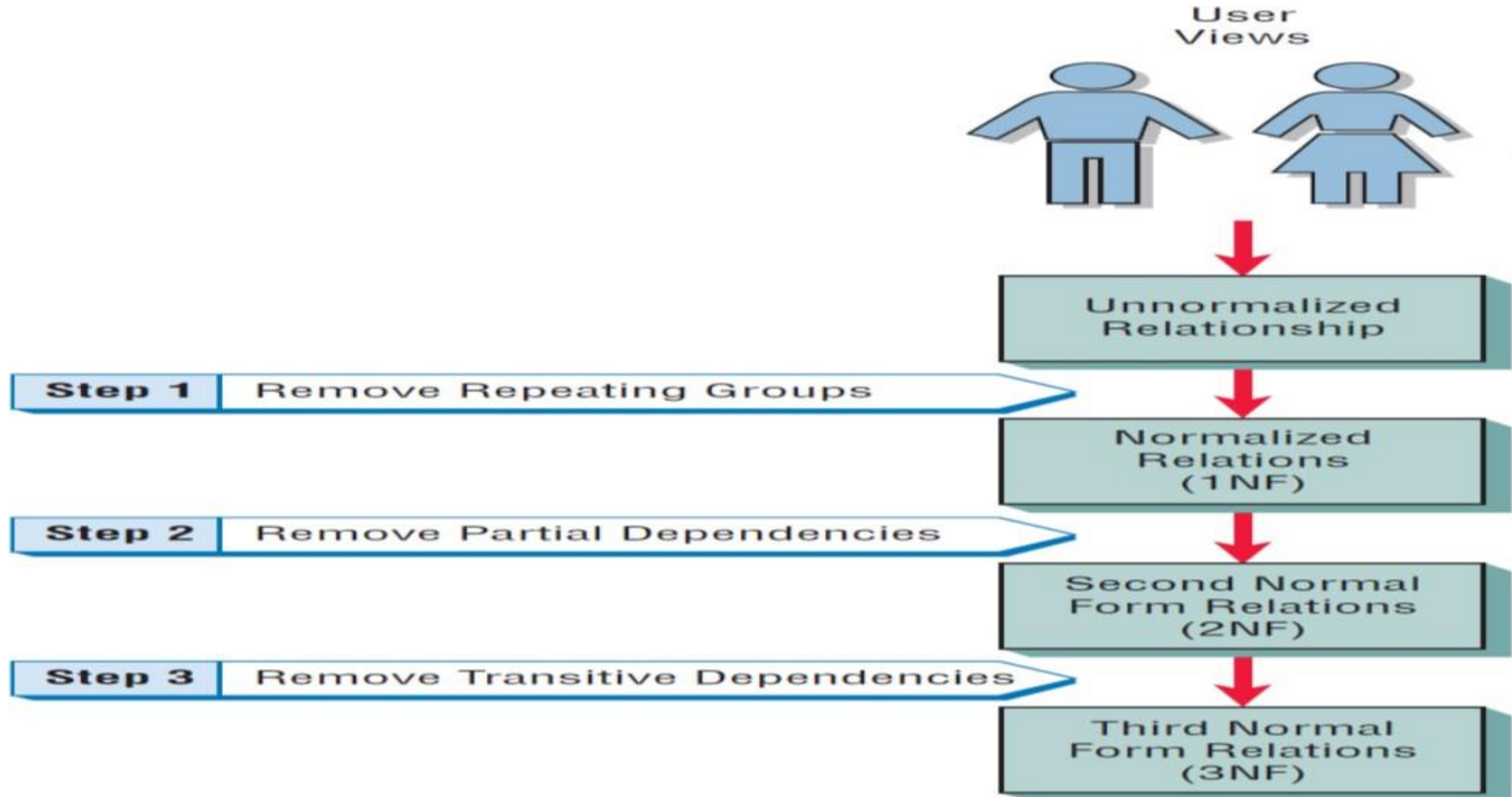
ED1



ED2

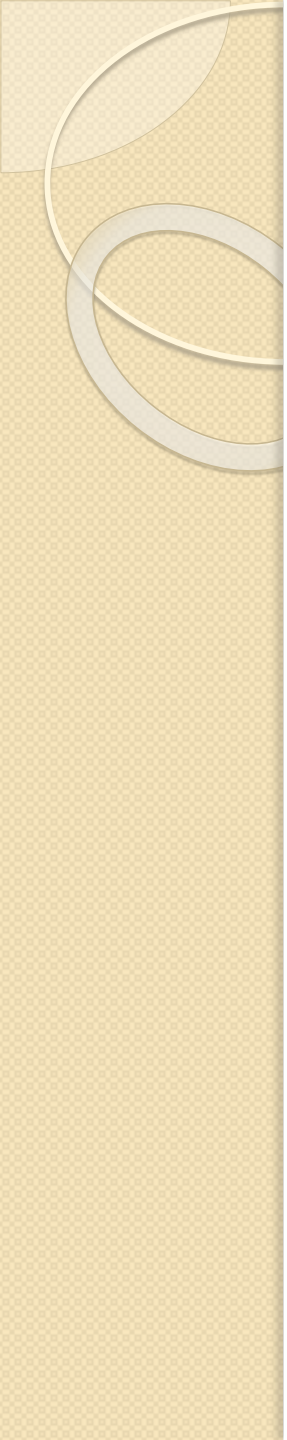


Normalization Summary



Cont....

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).



Thank you !!!